CÓMPUTO PARALELO Y EN LA NUBE

DBSCAN paralelo para detección de ruido

Descripción técnica y evaluación experimental individual

Proyecto: victormbr-creator/dbscan-openmp
Fecha de revisión: 19 de octubre de 2026
Entrega indicada: 20 de octubre de 2026

Alcance	Evidencia
Implementación	C++17: P0 serial, P1 OpenMP estático y P2 OpenMP por bloques dinámicos.
Evaluación	116 configuraciones, 10 repeticiones y 1160 mediciones válidas.
Resultado destacado	Speedup de 6.24 con P2, 200000 puntos, 10 hilos y 64 particiones.
Corrección	Ocho casos frente a scikit-learn y equivalencia contra P0 de las 116 configuraciones.

En mi proyecto clasifico puntos core de primer orden, segundo orden (borde) y ruido, sin asignar identificadores de clúster. En este informe reporto las mediciones de la campaña del 15 de octubre de 2026, sin mezclarlas con ejecuciones posteriores ni presentar estimaciones como mediciones.

Repositorio privado: github.com/victormbr-creator/dbscan-openmp. Los CSV de respaldo están en results/benchmarks/e5129e76b440779f/.

1. Algoritmo y datos

El vecindario de un punto contiene los puntos a distancia menor o igual que epsilon, incluido el propio punto. Un punto es core si su vecindario contiene al menos min_samples observaciones. Un no-core es de segundo orden si tiene algún core vecino; de lo contrario es ruido.

En la fase 1 recorro los posibles vecinos de cada punto y lo marco como core al alcanzar el umbral. En la fase 2 busco un core vecino para cada no-core. No permito que los bordes propaguen la condición de core. Utilizo las clases 0=ruido, 1=core y 2=segundo orden.

Comparo $dx^2 + dy^2 <= epsilon^2$, sin calcular raíz cuadrada. Detengo la búsqueda cuando alcanzo min_samples o encuentro el primer core vecino, según la fase. Determino la condición de umbral, no necesariamente el conteo total. Aplico estas optimizaciones en P0, P1 y P2.

Utilizo memoria O(N) y conservo un peor caso temporal $O(N^2)$, sin construir una matriz de distancias. La parada temprana reduce el trabajo en los datos densos estudiados; no generalizo esa reducción a otros conjuntos.

Generación reproducible

Parámetro	Valor
Generador	sklearn.datasets.make_blobs
centers / cluster_std	4 / 0.06
random_state / center_box	11 / (0, 1)
Transformación	Inversión de columnas, valor absoluto y redondeo a 3 decimales.
Clasificación	epsilon=0.03; min_samples=10
Formato	CSV sin encabezados, dos columnas numéricas x,y.

Con scripts/generar_datos.py determino la ruta data/ desde la ubicación del script, independientemente del directorio de ejecución. Verifico la forma, los valores finitos y la igualdad exacta al recargar. Mantengo los parámetros y transformaciones del notebook del profesor.

Comparto la lectura y las reglas en src/dbscan_common.hpp. Desde serial.cpp, paralelo1.cpp y paralelo2.cpp selecciono la estrategia. Utilizo scripts separados para generar, validar, medir, auditar y graficar.

2. Paralelización y alcance de P2

Versión	Distribución	Consulta de vecinos
P0	Bucles secuenciales	Todo el conjunto
P1	omp for schedule(static), sobre puntos	Todo el conjunto
P2	omp for schedule(dynamic,1), sobre k bloques de índices	Todo el conjunto

En P2 asigno al bloque b los índices desde floor(N*b/k) hasta floor(N*(b+1)/k), excluyendo el extremo final. Cubro todos los puntos sin superposición, sin crear copias de los datos ni particiones espaciales aisladas.

Asigno a cada índice un único escritor y mantengo los puntos de sólo lectura. Conservo la barrera implícita al terminar la fase 1 para completar el vector global core antes de comenzar la fase 2. Utilizo bytes independientes y evito vector<bool>.

Tratamiento de resultados entre particiones

Con la partición reparto el trabajo sin limitar el vecindario: al procesar un punto del bloque A puedo contar vecinos y consultar cores del bloque B. Integro las contribuciones directamente en posiciones disjuntas de los vectores compartidos. No implemento una fase independiente de fusión de clasificaciones locales ni reconciliación de clústeres.

Interpreto la diapositiva 11 como el requisito de dividir la matriz en k partes, procesarlas en paralelo y mantener consistencia al unir sus resultados, sin imponer una reconciliación concreta. Respondo mediante partición lógica de filas y composición directa por índices exclusivos; con consultas globales conservo los vecinos entre bloques. Presento esta explicación como mi justificación técnica, no como una aprobación expresa del profesor. La barrera sincroniza las fases, no fusiona clasificaciones locales.

Si necesitara una unión explícita, podría contar vecinos por pares de bloques y sumar las contribuciones antes de decidir cores; después combinaría mediante OR la existencia de cores vecinos. No implementé esa variante ni le atribuyo los resultados reportados: cambiaría el trabajo y requeriría validación y nuevas mediciones.

En P1 elegí reparto estático para reducir coordinación. En P2 utilizo reparto dinámico por bloques para ofrecer más oportunidades de balance. Si k es menor que la cantidad de hilos, sólo k hilos pueden procesar bloques simultáneamente. No instrumenté el tiempo por hilo para atribuir causalidad definitiva a las diferencias.

3. Entorno y diseño experimental

Elemento	Configuración registrada
Equipo	Apple M5; 10 procesadores físicos y 10 lógicos; 24 GiB RAM.
Sistema	macOS 26.5.2, build 25F84, arm64.
Compilador	Homebrew clang 23.1.3; OpenMP para P1/P2.
Opciones comunes	-std=c++17 -O3 -Wall -Wextra -Wpedantic -ffp-contract=off
Paquetes Python	NumPy 2.5.4; scikit-learn 1.9.1; Matplotlib 3.11.2.
N individual	20000, 80000, 140000, 200000.
Hilos T	1, 5, 10, 20.
Particiones k	2, 4, 8, 16, 32, 64.

En mi evaluación individual seleccioné cuatro de los ocho tamaños solicitados y mantuve todos los hilos y particiones. Por tamaño ejecuté 1 configuración P0 + 4 P1 + 24 P2 = 29. Obtuve 116 configuraciones y 1160 mediciones con diez repeticiones. Al usar 20 hilos introduje sobresuscripción sobre 10 procesadores lógicos.

Con steady_clock mido la inicialización de etiquetas, ambas fases, entrada al equipo OpenMP y sincronización. Excluyo lectura, conteo final de categorías, escritura CSV y salida JSON. Utilizo el mismo dataset para comparar las tres versiones.

Descarto un calentamiento por configuración e intercalo las diez repeticiones en orden pseudoaleatorio de semilla 11. Conservo tiempos individuales, medias y desviaciones estándar sin eliminar observaciones. Considero que la carga y la temperatura pueden afectar los tiempos.

Speedup = media(P0) / media(paralelo). Eficiencia = speedup / hilos. La eficiencia no equivale al porcentaje de uso de CPU. Variar hilos con N fijo evalúa escalabilidad fuerte; variar N no constituye por sí solo una prueba formal de escalabilidad débil.

4. Resultados y variabilidad

N	Versión	T / k	Media ms	DE ms	Speedup
20000	P0	1 / -	9.4525	0.1542	1.000
20000	P1	10 / -	1.9046	0.0743	4.963
20000	P2	10 / 64	1.6737	0.0798	5.648
80000	P0	1 / -	40.5661	0.6320	1.000
80000	P1	20 / -	7.5737	0.1274	5.356
80000	P2	10 / 64	6.5907	0.2365	6.155
140000	P0	1 / -	74.3209	1.9423	1.000
140000	P1	20 / -	13.5729	0.1850	5.476
140000	P2	10 / 64	11.9734	0.2083	6.207
200000	P0	1 / -	104.3382	3.1925	1.000
200000	P1	20 / -	19.0317	0.4159	5.482
200000	P2	10 / 64	16.7330	0.3900	6.235

Tabla 1. Mejores medias observadas por versión y tamaño. Cada fila resume diez repeticiones. DE es desviación estándar muestral, no intervalo de confianza. Selecciono la mejor configuración con un fin descriptivo y conservo todas las configuraciones en summary.csv.

Ambas versiones paralelas superaron 1.5 en los cuatro tamaños evaluados. P2 con 10 hilos y 64 bloques obtuvo la mejor media en todos. Para 200000 puntos, la mejor P2 redujo el tiempo aproximadamente 12.1% frente a la mejor P1. Su eficiencia fue 62.35%; la de P1 con 20 hilos fue 27.41%.

No generalizo la mejor configuración a otros equipos, distribuciones o parámetros. En estos datos, la alta densidad permite parada temprana; el crecimiento aproximadamente lineal observado no cambia la complejidad de peor caso.

5. Tiempo y escalabilidad

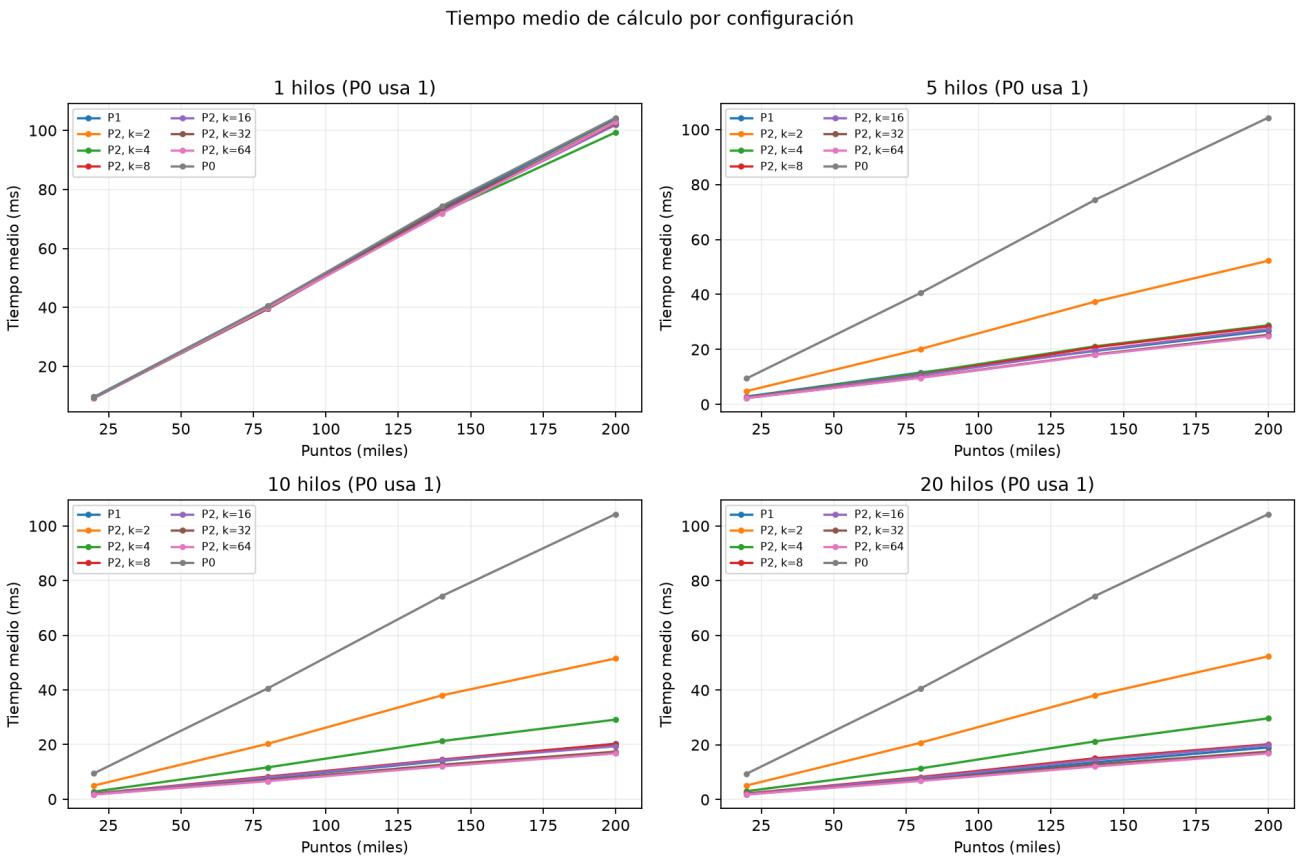


Figura 1. Tiempo medio de cálculo en milisegundos. Cada panel fija el número de hilos de las versiones paralelas; P0 siempre usa ejecución serial. Muestro todos los k de P2.

El tiempo aumenta con N. En P2, k=2 limita el trabajo concurrente a dos bloques y deja de aprovechar hilos adicionales. Con más bloques se observan menores tiempos. Las curvas corresponden a la clasificación y no incluyen entrada/salida.

Mantengo comparabilidad algorítmica: no utilizo un índice espacial exclusivo de una versión ni matrices precalculadas, y comparto las condiciones de parada.

6. Speedup frente a hilos

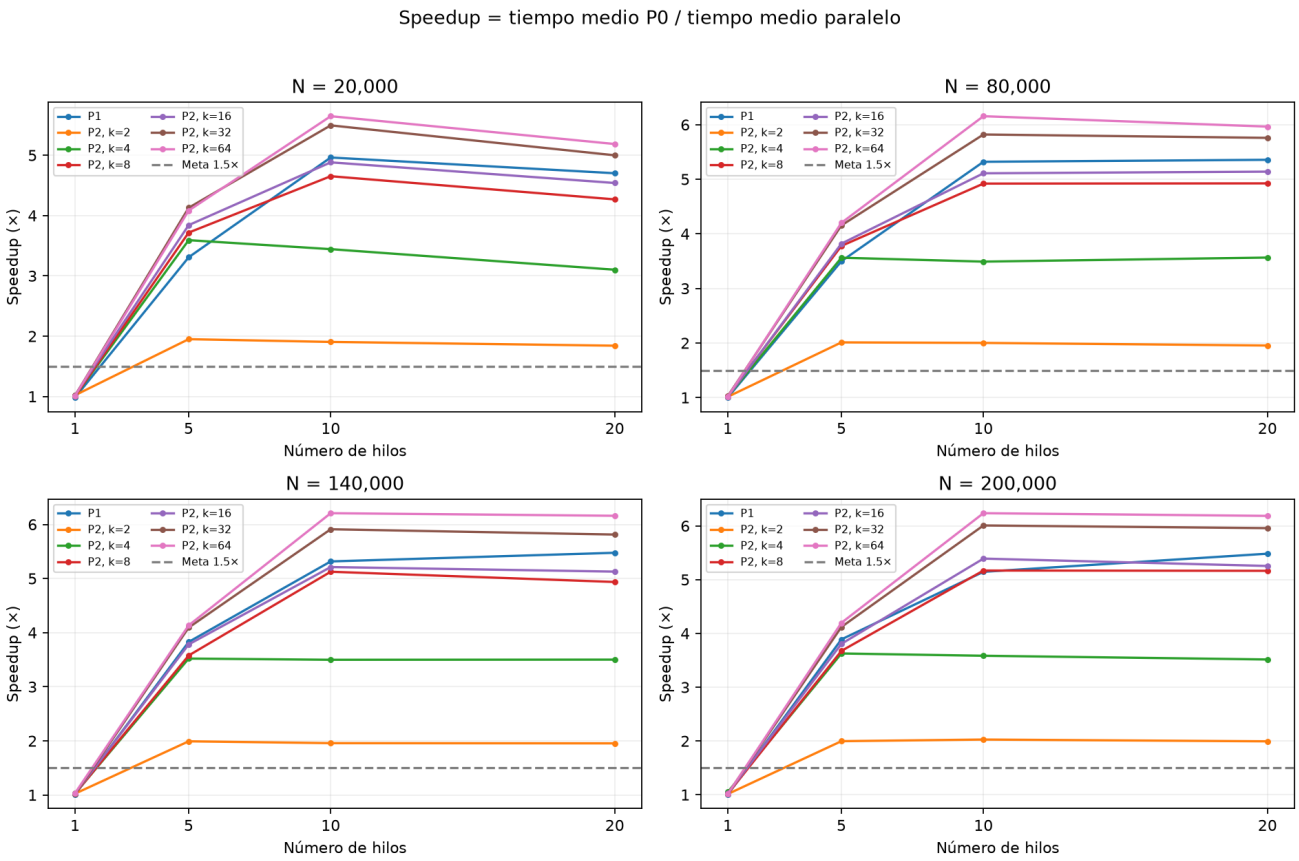


Figura 2. Speedup respecto a la media serial del mismo N. La línea discontinua marca el objetivo 1.5. Cada panel corresponde a un tamaño; las curvas de P2 conservan k fijo.

P2 alcanza su mejor media con 10 hilos y k=64. El paso a 20 hilos no mejora esa configuración. P1 alcanza mejores medias con 20 hilos en los tres tamaños mayores, pero no generalizo el efecto: no medí directamente planificación, temperatura ni desequilibrio por hilo.

7. Speedup frente a particiones

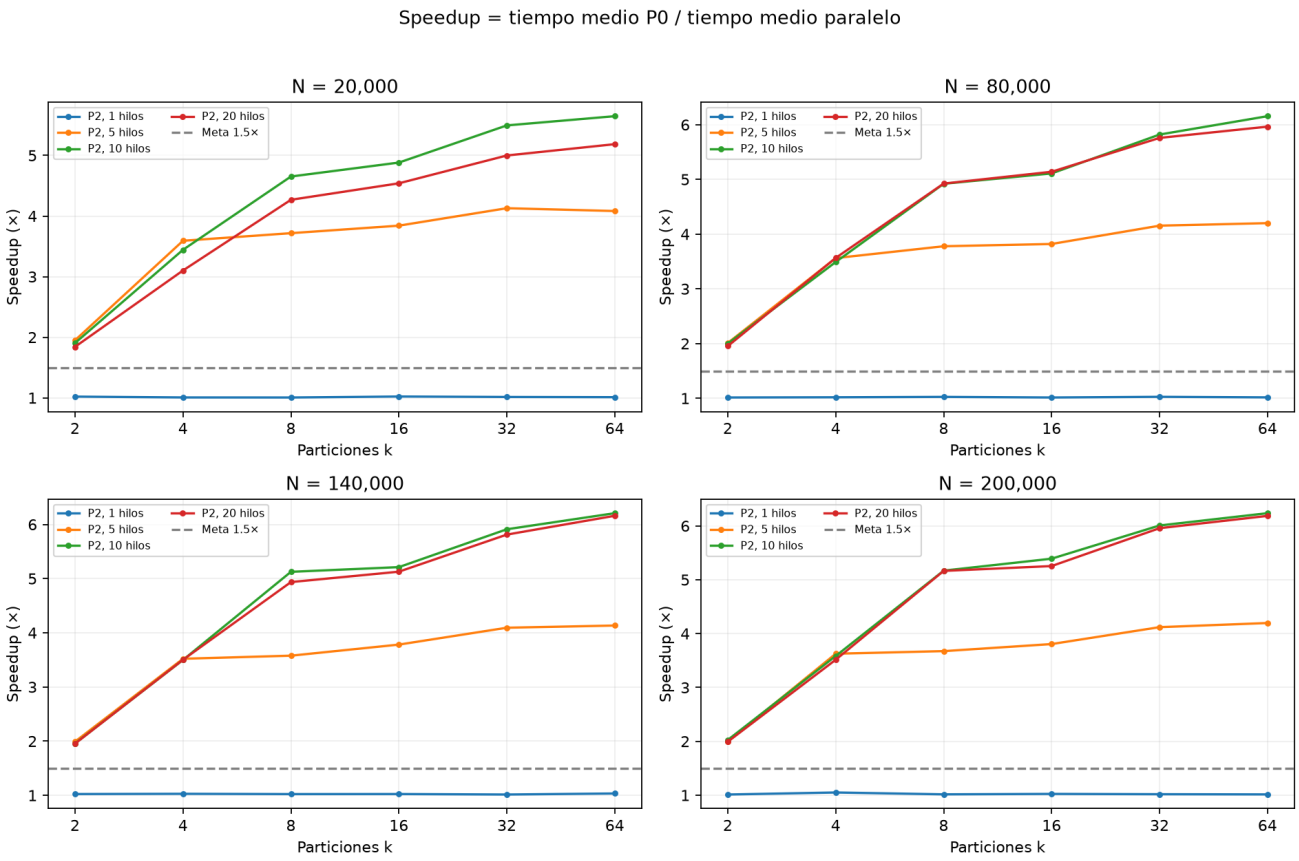


Figura 3. Speedup de P2 al variar k; el eje horizontal muestra potencias de dos equiespaciadas. Cada curva fija el número de hilos.

Con un hilo el speedup permanece cerca de uno. Con varios hilos, k=2 mantiene una aceleración cercana a dos. Aumentar k hasta 64 mejora el rendimiento medido con 10 hilos. Esto es compatible con mejor balance de carga, aunque no demuestra por sí solo su causa.

8. Validación, reproducción y uso de IA

Utilizo DBSCAN de scikit-learn con algorithm=kd_tree como referencia externa, core_sample_indices_ para cores y labels_=-1 para ruido. Clasifico como borde los no-core restantes. Probé datos del profesor, duplicados, distancia epsilon exacta, aislados, densos, un punto, min_samples mayor que N y vecinos entre bloques: 120 ejecuciones.

En la primera comparación con brute observé cinco discrepancias en 4000 puntos; kd_tree coincidió con mi implementación. Documenté la sensibilidad al cálculo flotante cerca del umbral, sin añadir tolerancias arbitrarias ni suponer identidad entre todos los backends.

Mediante la auditoría revisé hashes, diez repeticiones distintas por configuración, tiempos finitos y positivos, medias, speedup y eficiencia. Reejecuté las 116 configuraciones y comparé los CSV completos contra P0. Contrasté los datasets grandes con P0; no afirmo validación externa de cada uno con sklearn.

N	Core	Segundo orden	Ruido
20000	19866	89	45
80000	79885	81	34
140000	139896	69	35
200000	199902	77	21

Reproducción limpia

Con asistencia de Codex comprobé un clon remoto en una carpeta temporal, con un entorno virtual nuevo, dependencias instaladas y ejecutables y datos regenerados. Registro comandos, versión y resultados en docs/reproduccion_limpia.md. Utilicé el mismo Mac y las dependencias de sistema instaladas; no comprobé portabilidad a otro sistema operativo.

Utilicé Codex de OpenAI como apoyo en código, automatización, pruebas y documentación. Reporto resultados de ejecuciones reales y declaro la asistencia de IA. Realizo el proyecto individualmente y soy responsable de revisar las decisiones y comprender el código para mi defensa.

Archivos de respaldo: docs/metodologia.md, docs/resultados.md, docs/uso_ia.md y results/benchmarks/e5129e76b440779f/{raw.csv,summary.csv,metadata.json,validation.csv}. El PDF fue generado por scripts/generar_informe.py a partir de esos resultados.